

Syni: Acting on Human State Without Sensing It - An On-Device, Privacy-Oriented Agent over a Human State Interface

Israel Goytom Birhane

Human-AI Research Lab

Synheart AI Inc.

Vancouver, British Columbia, Canada

israel@synheart.ai

Yisak Tola Debele

Human-AI Research Lab

Synheart AI Inc.

Vancouver, British Columbia, Canada

yisak@synheart.ai

Henok Biadgign Ademtew

Human-AI Research Lab

Synheart AI Inc.

Vancouver, British Columbia, Canada

henok@synheart.ai

Anwar Misbah

Human-AI Research Lab

Synheart AI Inc.

Vancouver, British Columbia, Canada

anwar.misbah@synheart.ai

Abstract

Wearables expose signals about a person’s physiological, cognitive, and affective state, and large language models (LLMs) can turn them into personalized guidance. Existing systems typically run in the cloud and emit free-form text. We present **Syni**, an agent built on a strict separation: a deterministic upstream engine produces a versioned *Human State Interface* (HSI); Syni is purely its *consumer*. This lets Syni run offline—first on-device, condition through an audited seam, guarantee schema-safe output via grammar-constrained decoding with a generate–validate–repair–fallback pipeline, and keep raw biosignals off the model—simplifying deployment, validation, and privacy. In preliminary measurements, an *unversioned* renderer change between training and inference silently collapses state-reflection **from 85% to 0% while schema-validity stays 100%**—a failure no schema check catches. Grammar-constrained decoding lifts a 1B model’s first-pass validity from 0% to 100%, and coach inference runs on an iPhone 17 Pro in ~2 s—showing contract-mediated, on-device agents are practical.

CCS Concepts

• **Human-centered computing** → **Ubiquitous and mobile computing**; • **Computing methodologies** → *Natural language generation*.

Keywords

on-device LLM, wearables, human-state inference, constrained decoding, privacy, mobile interaction

ACM Reference Format:

Israel Goytom Birhane, Henok Biadgign Ademtew, Yisak Tola Debele, and Anwar Misbah. 2026. Syni: Acting on Human State Without Sensing It - An On-Device, Privacy-Oriented Agent over a Human State Interface. In *28th International Conference on Mobile Human-Computer Interaction*

(*MobileHCI '26*), August 31–September 03, 2026, Swansea, United Kingdom. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3821581.3833109>

1 Introduction

Mobile experiences increasingly aim to adapt to a person’s changing state—their focus, stress, or recovery—not merely what they tap. Continuous physiological and behavioral monitoring has made fine-grained estimates of a person’s state widely available [6, 14], and LLMs are an attractive interface to it [3, 5, 15]. Recent systems make this concrete: PHIA reasons over wearable data with code generation and retrieval [15], PH-LLM fine-tunes for sleep/fitness coaching [3], a multi-agent “personal health agent” spans diverse wellness needs [12], and PhysioLLM, in a controlled study, shows an LLM conversation over a week of wearable data produces insights users find more understandable and actionable than a generic chatbot [5].

Yet the dominant recipe has three structural limits for a production mobile system. It is *cloud-bound*: sensitive data is sent to a frontier model, complicating privacy and offline use. Generation is *unstructured*: free-form text is hard to validate, render, or bound to a safe envelope. And state inference and language generation are *entangled*, so the model is implicitly trusted to both interpret raw data and act on it.

We present **Syni**¹, an agent built on a simple principle: Syni is a *consumer* of the Human State Interface (HSI). A deterministic upstream engine ingests sensors, infers state, and emits HSI—a versioned JSON contract—which Syni takes as a typed input, leaving it responsible only for persona-driven, schema-safe generation, on-device and (with consent) in the cloud. This separation determines where privacy is enforced (upstream, before data reaches Syni), what Syni is tested against (a behavioral contract), and how the same agent runs identically offline and online.

Contributions. (1) An architecture that separates *measuring* human state (an upstream engine) from *acting on* it (Syni), with HSI as the only interface. (2) A *conditioning seam*: a versioned



This work is licensed under a Creative Commons Attribution 4.0 International License. *MobileHCI '26, Swansea, United Kingdom*

© 2026 Copyright held by the owner/author(s).

ACM ISBN /26/08

<https://doi.org/10.1145/3821581.3833109>

¹Open source: Syni’s cross-platform mobile bindings (Flutter, Swift, Kotlin) and the HSI contract are available at <https://github.com/synheart-ai/syni>, and <https://github.com/synheart-ai/hsi>; the on-device runtime is distributed as a free command-line install.

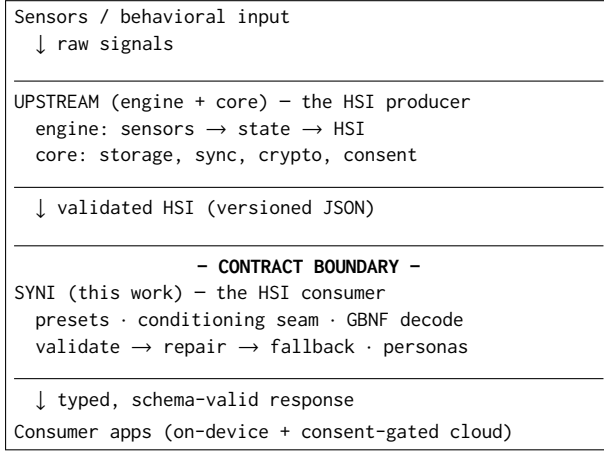


Figure 1: Syni consumes a versioned HSI contract and never touches sensors or raw signals; state inference and privacy enforcement live in the upstream producer.

rendering of HSI into the prompt, shared byte-identically by training and inference, and a measured demonstration that violating it silently degrades behavior. (3) Schema-safe on-device generation via grammar-constrained decoding plus a generate–validate–repair–fallback net, with an ablation isolating each stage’s contribution. (4) Preliminary on-device measurements from a deployed app on real iPhone and Android hardware, and a negative result on calibrated modesty. This is a systems contribution; user-facing benefits demonstrated by prior cloud work [5] are hypotheses we aim to *preserve* while adding on-device operation and a smaller data-exposure surface, not claims we measure here.

2 System

HSI (input contract). HSI is a well-defined, versioned JSON contract [16] (Fig. 1). It is the *interaction contract* between human-state inference systems and adaptive applications, letting state production and consumption evolve independently. At the source it carries five canonical axes—*physiological*, *kinematic*, *digital*, *cognitive*, and *affective*—each with readings, modality attribution, and optional confidence. Syni does not condition on all five directly: it conditions on a compact, task-relevant projection into the four dimensions a coaching response can actually influence—*focus*, *capacity*, *arousal*, and *recovery*—derived upstream from the canonical axes. This keeps the agent’s input aligned with what it can act on rather than the full sensing ontology. Producing HSI from raw signals is a separate, deterministic pipeline (out of scope here); Syni accepts any HSI version it advertises, treats unknown fields as forward-compatible, and never requests raw biosignals. The conditioning is a compact, confidence-gated rendering, e.g.:

```
Current human state:
- focus: 0.32 (low, confidence 0.60)
- capacity: 0.55 (moderate, confidence 0.70)
Treat these as estimates, not facts.
```

Table 1: Inference presets (latency/token/retry envelope).

preset	latency	tokens	retries
keyboard	150 ms	64	0
coach	2000 ms	256	2
chat	5000 ms	512	2

Local runtime. A native control-plane over llama.cpp [8] with a stable C ABI consumed by mobile SDKs. Syni targets small quantized models running locally (e.g. Gemma [7], MedGemma [9]), the regime that recent on-device foundation-model work [1] has made practical. Each request runs under a *preset*—a latency/token budget (Table 1)—that bounds worst-case on-device cost; sampling temperature also follows the preset (greedy for keyboard, cool for grounded coaching, warmer for chat). Orthogonal to presets, a *persona* is a fixed behavioral profile (tone, verbosity, coaching stance) selected per app or user that parameterizes generation: presets bound cost, personas shape voice.

Schema-safe generation. For each output schema the runtime emits a GBNF (GGML Backus–Naur Form) grammar that constrains decoding to exactly that JSON shape, so a small model can only emit schema-valid tokens. Grammar is the first line of defense, not the only one: output is parsed, validated against a JSON Schema, repaired (bounded structural fix) on failure, retried within budget, and—failing all—replaced by a deterministic, schema-valid fallback. The UI thus always receives a well-formed, typed response—well-formed, we stress, is not the same as correct (the grammar bounds structure, not content).

Conditioning seam. Syni renders the reduced HSI state into the system prompt. This rendering is a *contract*, not an implementation detail: a fine-tuned model learns a mapping from a *specific* rendering, so if the renderer used at training diverges from the one at inference, the model is evaluated out-of-distribution and degrades silently. We therefore pin the renderer with a single version tag and a cross-language golden test shared by the runtime and the fine-tuning pipeline. Following the modesty lesson from prior work [5], an axis is rendered only when it carries positive confidence—we never fabricate a band for a reading we do not have.

Privacy. Consent is enforced *upstream*: the core runtime applies tiered consent capabilities (raw biosignals, derived metrics, embeddings) before serialization, and the cloud rejects payloads exceeding the user’s tier. Syni may see validated HSI (or reduced summaries) and consented user text; it may *never* see raw R–R intervals, heart-rate-variability (HRV) waveforms, audio, or keystrokes. Because the model only receives a derived, consent-gated projection, the data-exposure surface is reduced by construction; keeping raw signals and free-form third-party content off the model also shrinks the surface for prompt injection [10] and training-data extraction [2]. We claim a *reduced model exposure surface*, not an end-to-end privacy guarantee. The load-bearing property—raw signals never reach the model—is enforced by the upstream trusted base (the core runtime’s consent gate and serializer). Syni consumes that base but does not verify it, so the guarantee holds only insofar as the base is correct. Concretely, the architecture prevents raw signals from reaching the model or the cloud whenever the upstream gate is correct. It

Table 2: GBNF ablation, schema-valid % (full budget). With grammar both models are 100% on every schema. Without it: first-pass / after-repair, for the 1B and 3B models. Repair alone cannot recover the strict keyboard/chat schemas (it makes bounded structural fixes, not free regeneration); only the grammar does.

schema	grammar on both	off, first-pass		off, +repair	
		1B	3B	1B	3B
keyboard	100	0	0	0	0
coach	100	0	100	100	100
chat	100	0	0	0	0

does *not* defend against an upstream producer that is itself buggy, misconfigured, or compromised: if the gate emits over-tier data, Syni has no independent check and will consume it. We harden the model’s input surface, not the producer itself. We do not measure privacy here.

3 Preliminary Evaluation

We present early measurements, not a controlled user study (appropriate to late-breaking work). System runs use gemma-3-1b-it (Q4_K_M) and Qwen2.5-3B-Instruct on an Apple M2 Max (Metal); fine-tuning uses QLoRA [4, 13] on an NVIDIA GPU; on-device numbers come from the deployed app (gemma-3-1b-it, Q4_K_M) on phones.

Grammar + repair isolation. With the grammar, both models are 100% first-pass schema-valid on every schema. Without it, at a generous budget, only the simplest schema (coach) reaches validity—the 3B model first-pass, the 1B via the repair step—while the stricter keyboard and chat schemas stay at 0% even after repair (Table 2). So the grammar’s marginal value concentrates on strict schemas and small models, exactly the on-device regime.

Conditioning-seam drift. To quantify the degradation a versioned seam prevents, we fine-tuned gemma-3-1b-it (QLoRA) on the canonical renderer, with targets written by a stronger teacher (so targets are natural language, not a rule), and evaluated the high-confidence subset of 64 held-out states under the matched renderer (A) vs. a drifted one (B). We deliberately make B a *worst case*—it changes three things at once (band words instead of numerics, a relabeled axis, and a single-line layout)—so the result is an upper bound on how badly an unversioned change can break the consumer, not a typical skew. State reflection—whether the response names the correct salient axis and band—is $85.3\% \pm 6.6$ under A and $0.0\% \pm 0.0$ under B (mean $\pm$ sd, 3 seeds), while schema-validity stays 100% on both (Table 3). Crucially, reflection is scored against the *ground-truth* salient axis and band of the underlying state—independent of how either renderer labels it—so B’s 0% reflects genuine loss of grounding, not a mechanical mismatch with relabeled tokens. The drifted output still parses and looks well-formed but no longer reflects the user’s state, so a schema check cannot catch it. A versioned, golden-tested seam removes this class of regression. Because B varies band words, axis labels, and layout together, it does not isolate which factor dominates; decomposing

Table 3: Conditioning-seam drift (gemma-3-1b-it, QLoRA, teacher targets; mean $\pm$ sd, 3 seeds). Drift collapses reflection while schema-validity is untouched.

renderer	state-reflection (%)	schema-valid (%)
A (train-matched)	85.3 ± 6.6	100
B (drifted)	0.0 ± 0.0	100

it into single-factor conditions is future work, though any one of them shipped unversioned is enough to motivate the invariant.

Calibrated modesty: a negative result. We asked whether the model learns to hedge *more* when state confidence is low—the modesty property cloud systems lack [5]. It does not, under realistic teacher targets: hedge rate is flat and non-monotone across the confidence grid (55/61/48/54/60/70% at confidence 0.1–0.9, mean over 3 seeds). A deterministic rule-target gives a clean step, but that merely memorizes the rule. Confidence-conditioned modesty did not transfer at the 1B scale; whether a stronger student or an explicit calibration objective closes the gap is open. (The hedge metric is a lexical proxy, not a reliability / expected calibration error (ECE) measure [11].)

On-device, real phones. The benefit of an HSI consumer is that it runs *on the device*. We benchmarked the coach preset from the deployed app on the current runtime, on two recent flagships ($N=20$ turns each, Table 4). The latency columns report first-pass decode; a turn *falls back* when the end-to-end turn—first pass plus any validate–repair retry—exceeds the budget or fails validation, in which case the net returns a deterministic valid response. On the iPhone (Metal-accelerated), a single pass already takes ~ 1.9 s—under, but close to, the 2 s coach budget—so any turn that needs even one retry overruns it; this is why 35% (7/20) fall back despite a sub-budget median. On the Android phone the same code runs unaccelerated (CPU) at ~ 11 s, always exceeding the budget, so *every* turn falls back to a guaranteed valid response—graceful degradation rather than a stall. Two findings stand out. (i) Enabling grammar-constrained decoding lowered the iPhone fallback rate relative to an earlier non-grammar build—informally, from roughly half of turns to the 35% reported here, though we did not run this as a controlled A/B—because valid first-pass output leaves fewer retries to blow the budget. (ii) The $\sim 6\times$ iPhone/Android gap on identical code is exactly why a latency-*bounding* contract—presets plus a fallback net—matters on heterogeneous mobile hardware: the user always receives a typed, schema-valid response, even when the model is too slow to finish in budget. That fallback is a fixed, state-neutral valid response, not a personalized one: it preserves structural safety and never stalls, but sacrifices personalization and can read as repetitive across a session—so on unaccelerated hardware the fallback rate, not latency alone, is the number to drive down. This gap is an acceleration artifact, not a fundamental limit: in preliminary follow-up measurements, GPU offload via Vulkan on the Android device cuts coach latency $\sim 4\times$ (median ~ 11.2 s to ~ 2.8 s), bringing it near the 2 s budget; closing the remaining gap through budget and quantization tuning is ongoing.

Table 4: On-device coach inference (gemma-3-1b-it Q4_K_M), deployed app, current runtime, $N=20$ turns. Latency is first-pass decode; *fallback* counts turns whose end-to-end time (with retries) overran the 2 s budget or failed validation. The iPhone (Metal) straddles the budget; the unaccelerated Android build always exceeds it and falls back to a valid response.

device	p_{50} (ms)	p_{95} (ms)	fallback (%)
iPhone 17 Pro (Metal)	1910	2039	35
Pixel 10 Pro (CPU)	11235	14695	100

4 Discussion and Conclusion

Interaction model. We do not yet evaluate the user-facing experience, but the architecture already fixes several of its properties. Syni surfaces guidance only within a preset’s latency budget and only for axes carrying positive confidence, so low-confidence state is omitted rather than guessed at, and every suggestion is framed to the user as an estimate, not a fact—mirroring the confidence-gated rendering the model sees. What the agent *cannot* yet do—let a user inspect the state estimate behind a suggestion, correct it, or veto a coaching moment—is exactly the interaction surface a user study should evaluate next. The contract-mediated design makes those controls tractable: the state is an explicit, typed object rather than a hidden activation.

Syni reframes wearable-plus-LLM guidance as a contract-mediated, on-device, schema-safe agent rather than a cloud prompt pipeline. The conditioning-seam invariant and the grammar/repair/fallback net are general and transferable; the exposure argument localizes raw-signal exclusion upstream, so raw waveforms reach neither the model nor the cloud. More broadly, little here is specific to human state: HSI is one instance of a more general, *contract-mediated* approach to adaptive AI, in which an agent consumes a typed, versioned interface rather than raw data—a pattern we expect to transfer beyond wearables. Limitations are real and we state them: we do not yet report a user study—a longitudinal, in-the-wild deployment is the natural next step and our planned evaluation; the learning experiments are small- N ; schema-safety guarantees well-formed not *correct* output; calibrated modesty does not yet transfer at the 1B scale; and the privacy guarantee is conditional on the upstream trusted base. Because the agent conditions coaching on affective and cognitive state, acting on an *erroneous* estimate is a wellbeing risk; we mitigate it by rendering only confidence-gated estimates (“treat these as estimates, not facts”) and by falling back to neutral, non-prescriptive content rather than a confident guess, but a content-level safety envelope on what the coach says remains future work. As an early snapshot, Syni shows that “infer upstream, act downstream” yields an on-device agent whose behavior stays reliable across a versioned state contract, with raw signals confined upstream.

References

- [1] Apple Machine Learning Research. 2025. Introducing the Third Generation of Apple Foundation Models. <https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models>. Accessed 2026-06-15.
- [2] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting Training Data from Large Language Models. In *30th USENIX Security Symposium*.
- [3] Justin Cosentino, Anastasiya Belyaeva, et al. 2025. A personal health large language model for sleep and fitness coaching. *Nature Medicine* 31 (2025), 3394–3403.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314 [cs.LG]
- [5] Cathy Mengying Fang, Valdemar Danry, Nathan Whitmore, Andria Bao, Andrew Hutchison, Cayden Pierce, and Pattie Maes. 2024. PhysioLLM: Supporting Personalized Health Insights with Wearables and Large Language Models. arXiv:2406.19283 [cs.HC]
- [6] Emilio Ferrara. 2024. Large Language Models for Wearable Sensor-Based Human Activity Recognition, Health Monitoring, and Behavioral Modeling: A Survey of Early Trends, Datasets, and Challenges. *Sensors* 24, 15 (2024), 5045.
- [7] Gemma Team, Google DeepMind. 2024. Gemma: Open Models Based on Gemini Research and Technology. arXiv:2403.08295 [cs.CL]
- [8] Georgi Gerganov and llama.cpp contributors. 2023. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>. GBNF grammar-constrained decoding.
- [9] Google DeepMind. 2025. MedGemma. <https://deepmind.google/models/gemma/medgemma/>. Accessed 2026-06-15.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173 [cs.CR]
- [11] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *International Conference on Machine Learning (ICML)*.
- [12] A. Ali Heydari, Ken Gu, Vidya Srinivas, Hong Yu, Zhihan Zhang, Yuwei Zhang, Akshay Paruchuri, et al. 2025. The Anatomy of a Personal Health Agent. arXiv:2508.20148 [cs.AI]
- [13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL]
- [14] Yubin Kim, Xuhai Xu, Daniel McDuff, Cynthia Breazeal, and Hae Won Park. 2024. Health-LLM: Large Language Models for Health Prediction via Wearable Sensor Data. arXiv:2401.06866 [cs.CL]
- [15] Mike A. Merrill, Akshay Paruchuri, Naghmeh Rezaei, Geza Kovacs, Javier Perez, Yun Liu, Erik Schenck, Nova Hammerquist, Jake Sunshine, Shyam Tailor, Kumar Ayush, Hao-Wei Su, Qian He, Cory Y. McLean, Mark Malhotra, Shwetak Patel, Jiening Zhan, Tim Althoff, Daniel McDuff, and Xin Liu. 2026. Transforming Wearable Data into Personal Health Insights using Large Language Model Agents. *Nature Communications* 17, 1 (2026), 1143.
- [16] Synheart AI Inc. 2025. The Human State Interface (HSI): A Canonical Contract for Interoperable Human-State Systems. Open specification (Apache-2.0), JSON Schema v1.3. <https://github.com/synheart-ai/hsi>.